

Android Tutorial

A Complete Guide

18 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:

<https://snehaliteng.github.io/tutorials/android/>

Getting Started with Android Development

What is Android?

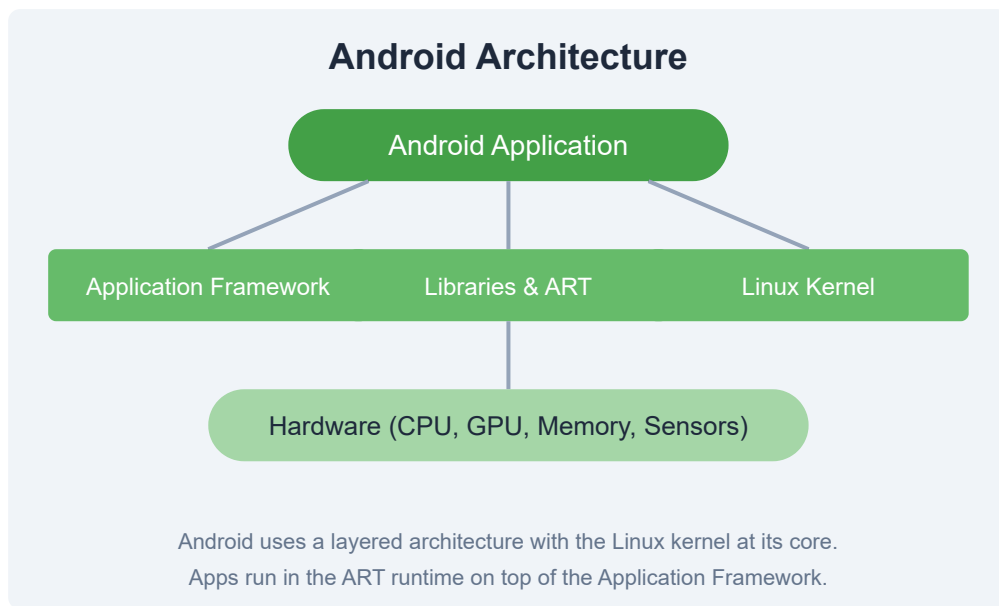
Android is an open-source mobile operating system based on the Linux kernel. Developed by Google, it powers billions of devices worldwide — phones, tablets, wearables, TVs, and even cars.

Key Fact: Android has over 3 billion active devices globally, making it the most widely used mobile operating system.

Why Develop for Android?

- **Massive User Base:** Reach billions of users across 190+ countries.
- **Open Ecosystem:** Publish on Google Play and alternative app stores.
- **Rich Capabilities:** Access to hardware (camera, GPS, sensors), notifications, background services.
- **Modern Tooling:** Android Studio, Kotlin, Jetpack Compose, and comprehensive SDK.
- **Monetization Options:** In-app purchases, subscriptions, ads, paid apps.

Android Architecture



Android's architecture consists of several layers:

1. **Linux Kernel:** The core OS layer providing drivers, memory management, and security.
2. **Hardware Abstraction Layer (HAL):** Interfaces between hardware and the framework.
3. **Android Runtime (ART):** Runs apps using ahead-of-time (AOT) compilation.
4. **Native C/C++ Libraries:** OpenGL, WebKit, SQLite, etc.
5. **Application Framework:** Java/Kotlin APIs for building apps.
6. **System Apps:** Built-in applications like Phone, Contacts, Browser.

Application Components

- **Activities:** Single screen with a user interface.
- **Services:** Background operations without a UI.
- **Broadcast Receivers:** Respond to system-wide events.
- **Content Providers:** Share data between apps.

Activity Lifecycle



Android Activity lifecycle stages

An Activity goes through several states:

- `onCreate()` — Called when the activity is first created.
- `onStart()` — The activity becomes visible to the user.
- `onResume()` — The activity starts interacting with the user.
- `onPause()` — Another activity is in the foreground.
- `onStop()` — The activity is no longer visible.
- `onDestroy()` — The activity is destroyed.

Hello World App (Kotlin)

```
// MainActivity.kt
package com.example.hello

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.TextView
import android.widget.Button

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val greetingText: TextView = findViewById(R.id.greetingText)
        val clickMeButton: Button = findViewById(R.id.clickMeButton)

        clickMeButton.setOnClickListener {
            greetingText.text = "Hello, Android!"
        }
    }
}
```

```

<!-- activity_main.xml -->
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/greetingText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome!"
        app:layout_constraintCenter="parent" />

    <Button
        android:id="@+id/clickMeButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click Me"
        app:layout_constraintBeneath="@id/greetingText" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

Exercise: Create a new Android project in Android Studio with an Empty Activity. Run it on the emulator, then modify the layout to include two TextViews and a Button that changes the second TextView's text when clicked.

Environment Setup

Prerequisites

- A computer with at least 8GB RAM (16GB recommended)
- 30GB of free disk space
- Java Development Kit (JDK) 17 or later
- Internet connection for downloading SDK components

Installing Android Studio

1. Download Android Studio from developer.android.com/studio
2. Run the installer and follow the setup wizard
3. Select "Standard" installation type to install the latest SDK
4. Choose a dark or light theme
5. Wait for the SDK components to download

Android SDK Manager

The SDK Manager lets you install and manage Android SDK platforms, build tools, and system images.

```
// Key SDK components to install:
// - Android SDK Platform (latest API level)
// - Android SDK Build-Tools
// - Android Emulator
// - Intel HAXM or Windows Hyper-V for emulator acceleration
// - System images for different API levels

// Via command line (sdkmanager):
sdkmanager "platforms;android-34"
sdkmanager "build-tools;34.0.0"
sdkmanager "system-images;android-34;google_apis;x86_64"
sdkmanager --licenses
```

Android Studio Overview

- **Project View:** Browse your project files and resources.
- **Code Editor:** Write Kotlin/Java with intelligent completion.
- **Layout Editor:** Design UIs visually with drag-and-drop.
- **Logcat:** View system and app logs in real-time.
- **Profiler:** Monitor CPU, memory, network, and energy usage.
- **AVD Manager:** Create and manage Android Virtual Devices (emulators).

Project Structure

```
MyAndroidApp/  
  app/  
    src/  
      main/  
        java/com/example/app/  # Source code (Kotlin/Java)  
        res/  
          drawable/             # Images and drawable resources  
          layout/               # XML layout files  
          mipmap/               # Launcher icons  
          values/               # Strings, colors, themes  
        AndroidManifest.xml     # App configuration  
      test/                     # Unit tests  
        androidTest/           # Instrumented tests  
      build.gradle.kts          # App-level build config  
    build.gradle.kts            # Project-level build config  
    settings.gradle.kts         # Project settings  
    gradle.properties           # Gradle properties
```

Build System (Gradle)

```
// app/build.gradle.kts
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
}

android {
    namespace = "com.example.myapp"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.example.myapp"
        minSdk = 24
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"
    }

    buildTypes {
        release {
            isMinifyEnabled = true
            proguardFiles(getDefaultProguardFile("proguard-android.txt"))
        }
    }
}

dependencies {
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.appcompat:appcompat:1.6.1")
    testImplementation("junit:junit:4.13.2")
}
```

Setting Up the Emulator

```
# Create an AVD (Android Virtual Device) via command line
avdmanager create avd -n Pixel6_API34 -k "system-images;android-34;google_api

# Start the emulator
emulator -avd Pixel6_API34

# Or use Android Studio:
# Tools > AVD Manager > Create Virtual Device
```

Exercise: Install Android Studio, create a new project with Kotlin, set up an emulator with API 34 (Pixel 6 profile), and run the default "Hello World" app on the emulator. Take a screenshot of the running app.

Application Architecture

Android App Architecture: MVVM

Modern Android development follows the MVVM (Model-View-ViewModel) architecture pattern. It separates concerns and makes apps testable and maintainable.

```

// Model - Data layer
data class User(
    val id: Int,
    val name: String,
    val email: String
)

class UserRepository(private val api: UserApi) {
    suspend fun getUsers(): List<User> = api.fetchUsers()
}

// ViewModel - Presentation logic
class UserViewModel(private val repo: UserRepository) : ViewModel() {
    private val _users = MutableLiveData<List<User>>()
    val users: LiveData<List<User>> = _users

    fun loadUsers() {
        viewModelScope.launch {
            try {
                _users.value = repo.getUsers()
            } catch (e: Exception) {
                _users.value = emptyList()
            }
        }
    }
}

// View - UI layer (Activity/Fragment)
class UserActivity : AppCompatActivity() {
    private lateinit var viewModel: UserViewModel

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_users)

        val repo = UserRepository(UserApi())
        viewModel = ViewModelProvider(this, UserViewModelFactory(repo))
            .get(UserViewModel::class.java)

        viewModel.users.observe(this) { users ->
            // Update RecyclerView adapter
        }

        viewModel.loadUsers()
    }
}

```

```
}  
}
```

Jetpack Components

Android Jetpack is a suite of libraries to help you follow best practices:

- **Lifecycle:** Manage activity/fragment lifecycle
- **LiveData:** Observable data holder
- **ViewModel:** Store UI-related data
- **Room:** SQLite abstraction
- **Navigation:** Simplify screen navigation
- **WorkManager:** Deferrable background tasks
- **Data Binding:** Bind UI components to data sources

Clean Architecture Layers

```
// Domain Layer - Business logic
interface UserRepository {
    suspend fun getUsers(): Result<List<User>>
}

class GetUsersUseCase(private val repo: UserRepository) {
    suspend operator fun invoke(): Result<List<User>> = repo.getUsers()
}

// Data Layer - Implementations
class UserRepositoryImpl(
    private val api: UserApi,
    private val dao: UserDao
) : UserRepository {
    override suspend fun getUsers(): Result<List<User>> = runCatching {
        val cached = dao.getAllUsers()
        if (cached.isNotEmpty()) cached
        else api.fetchUsers().also { dao.insertAll(it) }
    }
}

// Presentation Layer
@HiltViewModel
class UserViewModel @Inject constructor(
    private val getUsers: GetUsersUseCase
) : ViewModel() {
    val uiState: StateFlow<UiState<List<User>>> = ...
}
```

Dependency Injection with Hilt

```
// app/build.gradle.kts
plugins { id("com.google.dagger.hilt.android") }

dependencies {
    implementation("com.google.dagger:hilt-android:2.48")
    kapt("com.google.dagger:hilt-compiler:2.48")
}

// Application class
@HiltAndroidApp
class App : Application()

// Activity
@AndroidEntryPoint
class MainActivity : AppCompatActivity()
```

Navigation Component

```
<!-- res/navigation/nav_graph.xml -->
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    app:startDestination="@id/homeFragment">

    <fragment android:id="@+id/homeFragment"
        android:name=".HomeFragment">
        <action android:id="@+id/to_detail"
            app:destination="@id/detailFragment" />
    </fragment>

    <fragment android:id="@+id/detailFragment"
        android:name=".DetailFragment" />
</navigation>

// In code
findNavController().navigate(R.id.to_detail, bundleOf("id" to 123))
```


Exercise: Create an Android app that follows MVVM architecture. Use a ViewModel with LiveData to hold a list of items, a RecyclerView in the Activity to display them, and a Repository pattern to provide the data (use a mock data source). Add a loading state using LiveData.

Core Components

Activities

An Activity represents a single screen with a user interface. It's the entry point for interacting with the user.

```
// Manifest declaration
<activity android:name=".MainActivity">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>

// Activity with intent extras
class DetailActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_detail)

        val itemId = intent.getIntExtra("ITEM_ID", 0)
        val itemName = intent.getStringExtra("ITEM_NAME") ?: ""
    }
}

// Starting an activity
val intent = Intent(this, DetailActivity::class.java).apply {
    putExtra("ITEM_ID", 42)
    putExtra("ITEM_NAME", "Sample")
}
startActivity(intent)
```

Fragments

Fragments represent reusable portions of your app's UI. They have their own lifecycle and can be combined in multiple activities.

```
class HomeFragment : Fragment() {
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_home, container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        // Setup UI components
    }
}

// Adding a fragment to an activity
supportFragmentManager.beginTransaction()
    .replace(R.id.fragmentContainer, HomeFragment())
    .addToBackStack(null)
    .commit()
```

Services

```
class MusicService : Service() {
    private lateinit var mediaPlayer: MediaPlayer

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        when (intent?.action) {
            "PLAY" -> playMusic()
            "STOP" -> stopMusic()
        }
        return START_STICKY
    }

    private fun playMusic() {
        mediaPlayer = MediaPlayer.create(this, R.raw.song)
        mediaPlayer.start()
    }

    private fun stopMusic() {
        mediaPlayer?.stop()
        mediaPlayer?.release()
        stopSelf()
    }

    override fun onBind(intent: Intent?): IBinder? = null
}
```

Broadcast Receivers

```
class AirplaneModeReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action == Intent.ACTION_AIRPLANE_MODE_CHANGED) {
            val isEnabled = intent.getBooleanExtra("state", false)
            Toast.makeText(context,
                "Airplane Mode: ${if (isEnabled) "ON" else "OFF"}",
                Toast.LENGTH_SHORT).show()
        }
    }
}

// Register in manifest
<receiver android:name=".AirplaneModeReceiver">
    <intent-filter>
        <action android:name="android.intent.action.AIRPLANE_MODE" />
    </intent-filter>
</receiver>
```

Content Providers

```
class NoteProvider : ContentProvider() {
    private lateinit var dbHelper: SQLiteOpenHelper

    override fun query(uri: Uri, projection: Array<String>?, selection: String?,
        selectionArgs: Array<String>?, sortOrder: String?): Cursor? {
        val db = dbHelper.readableDatabase
        return db.query("notes", projection, selection, selectionArgs,
            null, null, sortOrder)
    }

    override fun insert(uri: Uri, values: ContentValues?): Uri? {
        val db = dbHelper.writableDatabase
        val id = db.insert("notes", null, values)
        return ContentUris.withAppendedId(uri, id)
    }
}
```

Intents & Filters

```
// Explicit intent - specific component
val intent = Intent(this, DetailActivity::class.java)

// Implicit intent - system resolves component
val webIntent = Intent(Intent.ACTION_VIEW, Uri.parse("https://android.com"))
startActivity(webIntent)

// Share intent
val shareIntent = Intent(Intent.ACTION_SEND).apply {
    type = "text/plain"
    putExtra(Intent.EXTRA_TEXT, "Check out this Android app!")
}
startActivity(Intent.createChooser(shareIntent, "Share via"))

// Intent filter in manifest
<activity android:name=".ShareReceiverActivity">
    <intent-filter>
        <action android:name="android.intent.action.SEND" />
        <category android:name="android.intent.category.DEFAULT" />
        <data android:mimeType="text/plain" />
    </intent-filter>
</activity>
```

Exercise: Create an Android app with two Activities. The first Activity has a list of items (mock data). Clicking an item opens the second Activity showing the item's details. Pass data using Intent extras. Add a Share button in the detail screen using an implicit intent.

UI Layouts

Layout Types

Android provides several layout managers to arrange UI elements:

ConstraintLayout (Recommended)

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/title"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:text="Welcome"
        android:textSize="24sp" />

    <Button
        android:id="@+id/submitBtn"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@id/title"
        app:layout_constraintStart_toStartOf="@id/title"
        app:layout_constraintEnd_toEndOf="@id/title"
        android:text="Submit" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

LinearLayout

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Name:" />

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your name" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Save"
        android:layout_gravity="end" />
</LinearLayout>
```

FrameLayout

Used to stack views on top of each other. Often used for fragments.

```
<FrameLayout
    android:id="@+id/fragmentContainer"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:foreground="?android:selectableItemBackground" />
```

Material Design Components

Use Material Design 3 (Material You) for modern Android UIs:


```
// app/build.gradle.kts
dependencies {
    implementation("com.google.android.material:material:1.11.0")
}

// XML theme
<style name="Theme.MyApp" parent="Theme.Material3.Light.NoActionBar">
    <item name="colorPrimary">@color/primary</item>
    <item name="colorSecondary">@color/secondary</item>
    <item name="colorTertiary">@color/tertiary</item>
    <item name="android:colorBackground">@color/background</item>
</style>

// Material Components
<com.google.android.material.button.MaterialButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    style="@style/Widget.Material3.Button.OutlinedButton"
    android:text="Outlined Button" />

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardCornerRadius="12dp"
    app:cardElevation="4dp">
    <!-- Card content -->
</com.google.android.material.card.MaterialCardView>
```

Responsive Layouts with SlidingPaneLayout

```
<androidx.slidingpanelayout.widget.SlidingPaneLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- List pane -->
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/listPane"
        android:layout_width="280dp"
        android:layout_height="match_parent" />

    <!-- Detail pane -->
    <FrameLayout
        android:id="@+id/detailPane"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</androidx.slidingpanelayout.widget.SlidingPaneLayout>
```

Best Practice: Always use `ConstraintLayout` for complex layouts. It creates a flat view hierarchy that performs better than nested `LinearLayouts`.

Exercise: Create a login screen layout using `ConstraintLayout` with the following elements: an `ImageView` (app logo) centered at the top, an `EditText` for email, an `EditText` for password, a `MaterialButton` for "Sign In", and a `TextView` for "Forgot Password?". Use proper constraints and Material Design styling.

UI Controls & Widgets

Basic Controls

TextView & EditText

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Username"
    android:textColor="@color/textPrimary"
    android:textSize="16sp" />

<com.google.android.material.textfield.TextInputLayout
    style="@style/Widget.Material3.TextInputLayout.OutlinedBox"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter your username">
    <com.google.android.material.textfield.TextInputEditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="text" />
</com.google.android.material.textfield.TextInputLayout>
```

Button Variants

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Default Button" />

<com.google.android.material.button.MaterialButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    style="@style/Widget.Material3.Button.TonalButton"
    android:text="Tonal" />

<com.google.android.material.button.MaterialButton
    style="@style/Widget.Material3.Button.TextButton"
    android:text="Text Button" />

<com.google.android.material.button.MaterialButton
    style="@style/Widget.Material3.Button.IconButton"
    android:icon="@drawable/ic_search" />

<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:id="@+id/fab"
    android:src="@drawable/ic_add" />
```

RecyclerView

```
// Layout
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager" />

// Adapter
class UserAdapter(private val users: List<User>) :
    RecyclerView.Adapter<UserAdapter.UserViewHolder>() {

    class UserViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val nameText: TextView = itemView.findViewById(R.id.nameText)
        val emailText: TextView = itemView.findViewById(R.id.emailText)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): UserViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_user, parent, false)
        return UserViewHolder(view)
    }

    override fun onBindViewHolder(holder: UserViewHolder, position: Int) {
        val user = users[position]
        holder.nameText.text = user.name
        holder.emailText.text = user.email
        holder.itemView.setOnClickListener {
            // Handle click
        }
    }

    override fun getItemCount() = users.size
}

// Setup in Activity
recyclerView.adapter = UserAdapter(users)
recyclerView.addItemDecoration(DividerItemDecoration(this, DividerItemDecorat
```

Different LayoutManagers

```
// Linear layout
recyclerView.layoutManager = LinearLayoutManager(this)

// Grid layout
recyclerView.layoutManager = GridLayoutManager(this, 2)

// Staggered grid (like Pinterest)
recyclerView.layoutManager = StaggeredGridLayoutManager(2,
    StaggeredGridLayoutManager.VERTICAL)
```

Other Important Widgets

```
<!-- ProgressBar -->
<ProgressBar
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:indeterminate="true" />

<!-- ProgressBar (horizontal) -->
<ProgressBar
    style="@style/Widget.AppCompat.ProgressBar.Horizontal"
    android:progress="65"
    android:max="100" />

<!-- SeekBar (slider) -->
<SeekBar
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:max="100"
    android:progress="50" />

<!-- Switch -->
<Switch
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Enable notifications" />

<!-- WebView -->
<WebView
    android:id="@+id/webview"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />

// WebView in code
webview.settings.javaScriptEnabled = true
webview.loadUrl("https://developer.android.com")
```

ViewPager2

```
<androidx.viewpager2.widget.ViewPager2
    android:id="@+id/viewPager"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />

// Fragment adapter
class OnboardingAdapter(activity: AppCompatActivity) :
    FragmentStateAdapter(activity) {
    override fun getItemCount() = 3
    override fun createFragment(position: Int): Fragment {
        return OnboardingFragment.newInstance(position)
    }
}
```

Exercise: Create a RecyclerView-based contact list app. Each item shows a contact name and phone number. Add a FloatingActionButton that shows a Snackbar when clicked. Use a GridLayoutManager with 2 columns and add dividers between items.

Event Handling

Event Listeners

Android uses event listeners to handle user interactions. The common pattern is to set a listener on a view.

```
// Lambda syntax (Kotlin)
val button: Button = findViewById(R.id.myButton)
button.setOnClickListener {
    Toast.makeText(this, "Button clicked!", Toast.LENGTH_SHORT).show()
}

// Anonymous inner class
button.setOnClickListener(object : View.OnClickListener {
    override fun onClick(v: View?) {
        // Handle click
    }
})

// Activity implements listener
class MainActivity : AppCompatActivity(), View.OnClickListener {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        findViewById<Button>(R.id.myButton).setOnClickListener(this)
    }

    override fun onClick(v: View?) {
        when (v?.id) {
            R.id.myButton -> handleButtonClick()
        }
    }
}
```

Touch Events

```
// Set touch listener on a view
view.setOnTouchListener { v, event ->
    when (event.action) {
        MotionEvent.ACTION_DOWN -> {
            println("Touch down at (${event.x}, ${event.y})")
            true
        }
        MotionEvent.ACTION_MOVE -> {
            println("Moving at (${event.x}, ${event.y})")
            true
        }
        MotionEvent.ACTION_UP -> {
            println("Touch up")
            true
        }
        else -> false
    }
}

// Custom View with touch handling
class DrawingView(context: Context, attrs: AttributeSet?) : View(context, attrs) {
    private val path = Path()
    private val paint = Paint().apply {
        color = Color.BLACK
        strokeWidth = 8f
        style = Paint.Style.STROKE
        strokeCap = Paint.Cap.ROUND
    }

    override fun onTouchEvent(event: MotionEvent): Boolean {
        when (event.action) {
            MotionEvent.ACTION_DOWN -> path.moveTo(event.x, event.y)
            MotionEvent.ACTION_MOVE -> path.lineTo(event.x, event.y)
            else -> return false
        }
        invalidate() // Redraw
        return true
    }

    override fun onDraw(canvas: Canvas) {
        canvas.drawPath(path, paint)
    }
}
```

```

    }
}

```

Gesture Detection

```

class GestureActivity : AppCompatActivity() {
    private lateinit var gestureDetector: GestureDetectorCompat

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        gestureDetector = GestureDetectorCompat(this, GestureListener())
    }

    override fun onTouchEvent(event: MotionEvent?): Boolean {
        return gestureDetector.onTouchEvent(event) || super.onTouchEvent(event)
    }

    inner class GestureListener : GestureDetector.SimpleOnGestureListener() {
        override fun onFling(
            e1: MotionEvent?, e2: MotionEvent?,
            velocityX: Float, velocityY: Float
        ): Boolean {
            val dx = e2?.x?.minus(e1?.x ?: 0f) ?: 0f
            if (Math.abs(dx) > 100 && Math.abs(velocityX) > 200) {
                if (dx > 0) {
                    Toast.makeText(this@GestureActivity, "Swiped Right", Toast.LENGTH_SHORT)
                } else {
                    Toast.makeText(this@GestureActivity, "Swiped Left", Toast.LENGTH_SHORT)
                }
                return true
            }
            return false
        }

        override fun onDoubleTap(e: MotionEvent?): Boolean {
            Toast.makeText(this@GestureActivity, "Double Tap", Toast.LENGTH_SHORT)
            return true
        }
    }
}

```

Keyboard and Focus

```
// Show keyboard
val imm = getSystemService(Context.INPUT_METHOD_SERVICE) as InputMethodManager
imm.showSoftInput(editText, InputMethodManager.SHOW_IMPLICIT)

// Hide keyboard
imm.hideSoftInputFromWindow(currentFocus?.windowToken, 0)

// Handle keyboard enter action
editText.setOnEditorActionListener { v, actionId, event ->
    if (actionId == EditorInfo.IME_ACTION_DONE) {
        handleSearch(editText.text.toString())
        true
    } else false
}

// Listen for focus changes
editText.setOnFocusChangeListener { v, hasFocus ->
    if (hasFocus) {
        // EditText gained focus
    } else {
        // EditText lost focus
    }
}
```

Drag & Drop

```
// Start drag
imageView.setOnLongClickListener { v ->
    val data = ClipData.newPlainText("image", "dragged_image")
    v.startDragAndDrop(data, View.DragShadowBuilder(v), null, 0)
    true
}

// Receive drop
targetView.setOnDragListener { v, event ->
    when (event.action) {
        DragEvent.ACTION_DRAG_STARTED -> {
            v.alpha = 0.5f
            true
        }
        DragEvent.ACTION_DROP -> {
            v.alpha = 1.0f
            // Handle drop
            Toast.makeText(context, "Item dropped!", Toast.LENGTH_SHORT).show()
            true
        }
        DragEvent.ACTION_DRAG_ENDED -> {
            v.alpha = 1.0f
            true
        }
        else -> true
    }
}
```

Exercise: Create a drawing app using custom View and touch events. The user should be able to draw with their finger, change the stroke color via a SeekBar, and clear the canvas with a button. Add swipe detection to show a "Clear?" confirmation dialog.

Resources & Assets

Resource Types

Android resources are non-code files organized in the `res/` directory. They are accessed via the `R` class.

Directory	Purpose
res/drawable/	Images, shapes, vector drawables, selectors
res/values/	Strings, colors, dimensions, themes, styles
res/layout/	XML layout files
res/mipmap/	Launcher icons (multiple densities)
res/font/	Custom font files (TTF, OTF)
res/raw/	Raw files (audio, video, JSON)

String Resources

```
<!-- res/values/strings.xml -->
<resources>
    <string name="app_name">My Android App</string>
    <string name="welcome_message">Welcome, %1$s!</string>
    <string name="item_count">%d items remaining</string>
    <string-array name="months">
        <item>January</item>
        <item>February</item>
        <item>March</item>
    </string-array>
</resources>

// Access in code
getString(R.string.welcome_message, "Alice")
resources.getStringArray(R.array.months)

// Access in XML
android:text="@string/welcome_message"
android:text="@string/welcome_message"
```

Color Resources

```
<!-- res/values/colors.xml -->
<resources>
    <color name="primary">#FF6200EE</color>
    <color name="primaryDark">#FF3700B3</color>
    <color name="accent">#FF03DAC5</color>
    <color name="background">#FFFFFFFF</color>
    <color name="textPrimary">#FF1C1B1F</color>
    <color name="textSecondary">#FF49454F</color>
</resources>

// Access
ContextCompat.getColor(this, R.color.primary)
android:background="@color/background"
```

Dimens Resources

```
<!-- res/values/dimens.xml -->
<resources>
    <dimen name="padding_small">8dp</dimen>
    <dimen name="padding_medium">16dp</dimen>
    <dimen name="padding_large">24dp</dimen>
    <dimen name="text_size_title">20sp</dimen>
    <dimen name="text_size_body">14sp</dimen>
    <dimen name="elevation_card">4dp</dimen>
</resources>

// Access
resources.getDimension(R.dimen.padding_medium)
android:padding="@dimen/padding_medium"
```

Themes & Styles

```
<!-- res/values/themes.xml (Material 3) -->
<resources>
    <style name="Theme.MyApp" parent="Theme.Material3.DayNight.NoActionBar">
        <item name="colorPrimary">@color/primary</item>
        <item name="colorOnPrimary">@color/white</item>
        <item name="colorSecondary">@color/secondary</item>
        <item name="android:colorBackground">@color/background</item>
        <item name="colorSurface">@color/surface</item>
        <item name="colorOnSurface">@color/textPrimary</item>
    </style>

    <style name="Widget.MyApp.Button" parent="Widget.Material3.Button">
        <item name="cornerRadius">12dp</item>
        <item name="iconTint">@color/white</item>
    </style>
</resources>
```


Drawable Resources

```
<!-- res/drawable/ic_search.xml (vector drawable) -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="?attr/colorControlNormal"
        android:pathData="M15.5,14h-0.79l-0.28,-0.27C15.41,12.59 16,11.11 16,
</vector>

// Shape drawable
<!-- res/drawable/rounded_button.xml -->
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="@color/primary" />
    <corners android:radius="24dp" />
    <padding android:left="16dp" android:right="16dp"
        android:top="8dp" android:bottom="8dp" />
</shape>
```

Raw Resources

```
// res/raw/config.json
InputStream stream = resources.openRawResource(R.raw.config);
BufferedReader reader = new BufferedReader(new InputStreamReader(stream));
val jsonString = reader.readText()

// Access in code
R.raw.sound_effect // Returns resource ID
```

Exercise: Create a themed Android app with custom colors, a vector drawable icon, custom fonts in `res/font/`, and string resources supporting English and Spanish (hint: use `res/values-es/strings.xml` for Spanish localization). Apply the theme globally in the manifest.

Advanced Concepts

Notifications

```
// Create notification channel (Android 8+)
val channel = NotificationChannel(
    "general",
    "General Notifications",
    NotificationManager.IMPORTANCE_DEFAULT
).apply {
    description = "General app notifications"
    enableVibration(true)
}

val notificationManager = getSystemService(NotificationManager::class.java)
notificationManager.createNotificationChannel(channel)

// Build notification
val notification = NotificationCompat.Builder(this, "general")
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("New Message")
    .setContentText("You have a new message from Alice")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .setAutoCancel(true)
    .addAction(R.drawable.ic_reply, "Reply", pendingIntent)
    .build()

// Show notification
notificationManager.notify(1001, notification)
```

Location Services

```
// Add dependency: implementation("com.google.android.gms:play-services-location:17.0.0")

val fusedLocationClient = LocationServices.getFusedLocationProviderClient(this)

// Request permission first
if (checkSelfPermission(Manifest.permission.ACCESS_FINE_LOCATION) == PackageManager.PERMISSION_GRANTED) {
    fusedLocationClient.lastLocation.addOnSuccessListener { location ->
        if (location != null) {
            val lat = location.latitude
            val lng = location.longitude
            println("Location: $lat, $lng")
        }
    }
}

// Request location updates
val locationRequest = LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY)
    .setMinUpdateIntervalMillis(5000)
    .build()

fusedLocationClient.requestLocationUpdates(locationRequest, locationCallback,
```

Sending Email & SMS

```
// Send email via Intent
fun sendEmail(to: String, subject: String, body: String) {
    val intent = Intent(Intent.ACTION_SENDTO).apply {
        data = Uri.parse("mailto:")
        putExtra(Intent.EXTRA_EMAIL, arrayOf(to))
        putExtra(Intent.EXTRA_SUBJECT, subject)
        putExtra(Intent.EXTRA_TEXT, body)
    }
    if (intent.resolveActivity(packageManager) != null) {
        startActivity(intent)
    }
}

// Send SMS
fun sendSMS(phoneNumber: String, message: String) {
    val intent = Intent(Intent.ACTION_SENDTO).apply {
        data = Uri.parse("smsto:$phoneNumber")
        putExtra("sms_body", message)
    }
    if (intent.resolveActivity(packageManager) != null) {
        startActivity(intent)
    }
}
```

Dialogs

```
// AlertDialog
AlertDialog.Builder(this)
    .setTitle("Delete Item")
    .setMessage("Are you sure you want to delete this item?")
    .setIcon(R.drawable.ic_warning)
    .setPositiveButton("Delete") { _, _ -> deleteItem() }
    .setNegativeButton("Cancel", null)
    .show()

// BottomSheetDialog
BottomSheetDialog(this).apply {
    setContentView(R.layout.dialog_bottom_sheet)
    show()
}

// Custom Dialog Fragment
class CustomDialogFragment : DialogFragment() {
    override fun onCreateDialog(savedInstanceState: Bundle?): Dialog {
        return AlertDialog.Builder(requireActivity())
            .setView(layoutInflater.inflate(R.layout.dialog_custom, null))
            .create()
    }
}
```

Firebase Integration

```
// build.gradle.kts
// implementation(platform("com.google.firebase:firebase-bom:32.7.0"))
// implementation("com.google.firebase:firebase-analytics")
// implementation("com.google.firebase:firebase-auth")
// implementation("com.google.firebase:firebase-firestore")

// Firebase Authentication
fun signInWithEmail(email: String, password: String) {
    Firebase.auth.signInWithEmailAndPassword(email, password)
        .addOnSuccessListener { println("Signed in") }
        .addOnFailureListener { println("Error: ${it.message}") }
}

// Firestore CRUD
val db = Firebase.firestore
db.collection("users").document("alice")
    .set(hashMapOf("name" to "Alice", "age" to 30))
    .addOnSuccessListener { println("Document saved") }

db.collection("users").get()
    .addOnSuccessListener { result ->
        for (document in result) {
            println("${document.id} => ${document.data}")
        }
    }
}
```

Exercise: Create an app that shows a notification with a "View" action button that opens the app. Add location tracking that updates a TextView with the current coordinates every 30 seconds (handle permissions gracefully). Show a confirmation dialog before sending analytics data.

Services & Background Tasks

Started Services

A Started Service runs in the background indefinitely, even if the activity that started it is destroyed.

```
class DownloadService : Service() {
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        val url = intent?.getStringExtra("URL") ?: return START_NOT_STICKY

        thread {
            downloadFile(url)
            stopSelf()
        }

        return START_STICKY // Restart if killed
    }

    private fun downloadFile(url: String) {
        // Download logic
        sendBroadcast(Intent("DOWNLOAD_COMPLETE"))
    }

    override fun onBind(intent: Intent?): IBinder? = null
}

// Start service
startService(Intent(this, DownloadService::class.java).apply {
    putExtra("URL", "https://example.com/file.zip")
})
```


Bound Services

```
class MusicService : Service() {
    private val binder = LocalBinder()
    private var mediaPlayer: MediaPlayer? = null

    inner class LocalBinder : Binder() {
        fun getService(): MusicService = this@MusicService
    }

    override fun onBind(intent: Intent?): IBinder = binder

    fun playMusic(song: Int) {
        mediaPlayer = MediaPlayer.create(this, song)
        mediaPlayer?.start()
    }

    fun stopMusic() {
        mediaPlayer?.stop()
        mediaPlayer?.release()
        mediaPlayer = null
    }
}

// Activity binding
class MainActivity : AppCompatActivity() {
    private var musicService: MusicService? = null
    private val connection = object : ServiceConnection {
        override fun onServiceConnected(name: ComponentName?, service: IBinder) {
            musicService = (service as MusicService.LocalBinder).getService()
        }

        override fun onServiceDisconnected(name: ComponentName?) {
            musicService = null
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        bindService(Intent(this, MusicService::class.java), connection, Context.BIND_AUTO_CREATE)
    }
}
```

Foreground Services

```
class ForegroundLocationService : Service() {
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        val notification = NotificationCompat.Builder(this, "location")
            .setContentTitle("Location Tracking")
            .setContentText("Tracking your location in background")
            .setSmallIcon(R.drawable.ic_location)
            .setOngoing(true)
            .build()

        startForeground(1002, notification)
        // Start location updates
        return START_STICKY
    }
}

// AndroidManifest.xml
<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
<service android:name=".ForegroundLocationService"
    android:foregroundServiceType="location" />
```

WorkManager

WorkManager is the recommended solution for deferrable background work that needs guaranteed execution.

```

// Define worker
class SyncWorker(context: Context, params: WorkerParameters) : Worker(context) {
    override fun doWork(): Result {
        return try {
            // Sync data to server
            syncData()
            Result.success()
        } catch (e: Exception) {
            Result.retry()
        }
    }
}

// Schedule one-time work
val syncRequest = OneTimeWorkRequestBuilder<SyncWorker>()
    .setConstraints(Constraints.Builder()
        .setRequiredNetworkType(NetworkType.CONNECTED)
        .setRequiresBatteryNotLow(true)
        .build())
    .setBackoffCriteria(BackoffPolicy.LINEAR, 10, TimeUnit.SECONDS)
    .build()

WorkManager.getInstance(this).enqueue(syncRequest)

// Observe work status
WorkManager.getInstance(this).getWorkInfoByIdLiveData(syncRequest.id)
    .observe(this) { info ->
        when (info?.state) {
            WorkInfo.State.SUCCEEDED -> println("Sync successful")
            WorkInfo.State.FAILED -> println("Sync failed")
        }
    }

// Periodic work
val periodicSync = PeriodicWorkRequestBuilder<SyncWorker>(15, TimeUnit.MINUTE)
    .build()
WorkManager.getInstance(this).enqueue(periodicSync)

```

JobScheduler

```
val jobScheduler = getSystemService(Context.JOB_SCHEDULER_SERVICE) as JobScheduler

val job = JobInfo.Builder(123, ComponentName(this, MyJobService::class.java))
    .setMinimumLatency(1000) // 1 second delay
    .setOverrideDeadline(5000) // Must run within 5 seconds
    .setRequiredNetworkType(JobInfo.NETWORK_TYPE_ANY)
    .setRequiresCharging(false)
    .build()

jobScheduler.schedule(job)
```

Exercise: Create a podcast app that uses a Foreground Service to play audio even when the app is in the background. Show a persistent notification with play/pause and stop controls. Use WorkManager to periodically check for new episodes every hour when the device is connected to Wi-Fi.

Storage & Data Persistence

SharedPreferences

```
// Save data
val prefs = getSharedPreferences("app_prefs", Context.MODE_PRIVATE)
prefs.edit().apply {
    putString("username", "Alice")
    putInt("age", 30)
    putBoolean("isLoggedIn", true)
    apply() // Async
}

// Read data
val username = prefs.getString("username", "")
val isLoggedIn = prefs.getBoolean("isLoggedIn", false)

// DataStore (modern replacement)
val datastore = context.createDataStore(name = "settings")
val EXAMPLE_COUNTER = intPreferencesKey("example_counter")
val exampleCounterFlow: Flow<Int> = datastore.data.map { it[EXAMPLE_COUNTER]

suspend fun incrementCounter() {
    datastore.edit { it[EXAMPLE_COUNTER] = (it[EXAMPLE_COUNTER] ?: 0) + 1 }
}
```

SQLite with Room

Room is the recommended abstraction layer over SQLite.

```

// Entity
@Entity(tableName = "notes")
data class Note(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    val content: String,
    val timestamp: Long = System.currentTimeMillis()
)

// DAO
@Dao
interface NoteDao {
    @Query("SELECT * FROM notes ORDER BY timestamp DESC")
    fun getAllNotes(): LiveData<List<Note>>

    @Insert
    suspend fun insert(note: Note)

    @Update
    suspend fun update(note: Note)

    @Delete
    suspend fun delete(note: Note)

    @Query("SELECT * FROM notes WHERE id = :id")
    suspend fun getNoteById(id: Int): Note?
}

// Database
@Database(entities = [Note::class], version = 1)
abstract class NoteDatabase : RoomDatabase() {
    abstract fun noteDao(): NoteDao

    companion object {
        @Volatile
        private var INSTANCE: NoteDatabase? = null

        fun getInstance(context: Context): NoteDatabase {
            return INSTANCE ?: synchronized(this) {
                Room.databaseBuilder(
                    context.applicationContext,
                    NoteDatabase::class.java,
                    "note_database"
                ).build().also { INSTANCE = it }
            }
        }
    }
}

```

```

        }
    }
}

// Usage
val dao = NoteDatabase.getInstance(this).noteDao()
viewModelScope.launch {
    dao.insert(Note(title = "My Note", content = "Hello Room!"))
}

```

File Storage

```

// Internal storage (private to the app)
fun writeToInternalFile(filename: String, content: String) {
    openFileOutput(filename, Context.MODE_PRIVATE).use {
        it.write(content.toByteArray())
    }
}

fun readFromInternalFile(filename: String): String {
    return openFileInput(filename).bufferedReader().readText()
}

// External storage (requires permission for public files)
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion="28" />

fun saveToDownloads(content: String, filename: String) {
    val downloadsDir = Environment.getExternalStoragePublicDirectory(
        Environment.DIRECTORY_DOWNLOADS
    )
    val file = File(downloadsDir, filename)
    file.writeText(content)
}

// Scoped storage (Android 10+)
fun saveToAppSpecificExternal() {
    val file = File(context.getExternalFilesDir(Environment.DIRECTORY_DOCUMENTS),
        "Hello, scoped storage!")
    file.writeText("Hello, scoped storage!")
}

```

JSON & XML Parsing

```
// JSON with Gson/Moshi
// implementation("com.squareup.moshi:moshi-kotlin:1.15.0")

@JsonClass(generateAdapter = true)
data class User(val id: Int, val name: String, val email: String)

val moshi = Moshi.Builder().build()
val adapter = moshi.adapter(User::class.java)

// Serialize
val user = User(1, "Alice", "alice@example.com")
val json = adapter.toJson(user)

// Deserialize
val userFromJson = adapter.fromJson(json)

// XML Pull Parser
fun parseXml(xml: String) {
    val parser = XmlPullParserFactory.newInstance().newPullParser()
    parser.setInput(StringReader(xml))
    var eventType = parser.eventType
    while (eventType != XmlPullParser.END_DOCUMENT) {
        if (eventType == XmlPullParser.START_TAG && parser.name == "item") {
            val name = parser.getAttributeValue(null, "name")
            println("Item: $name")
        }
        eventType = parser.next()
    }
}
```

Exercise: Create a notes app using Room database. Users can add, edit, and delete notes. Each note has a title, content, and timestamp. Display notes in a RecyclerView sorted by most recent. Show a dialog to confirm before deleting. Persist the last opened note ID using DataStore.

Networking & APIs

HTTP Clients

Retrofit (Recommended)

```
// build.gradle.kts
// implementation("com.squareup.retrofit2:retrofit:2.9.0")
// implementation("com.squareup.retrofit2:converter-gson:2.9.0")
// implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")

data class Post(
    val userId: Int,
    val id: Int,
    val title: String,
    val body: String
)

interface ApiService {
    @GET("posts")
    suspend fun getPosts(): List<Post>

    @GET("posts/{id}")
    suspend fun getPost(@Path("id") id: Int): Post

    @POST("posts")
    suspend fun createPost(@Body post: Post): Post

    @PUT("posts/{id}")
    suspend fun updatePost(@Path("id") id: Int, @Body post: Post): Post

    @DELETE("posts/{id}")
    suspend fun deletePost(@Path("id") id: Int)
}

// Retrofit instance
val logging = HttpLoggingInterceptor().apply {
    level = HttpLoggingInterceptor.Level.BODY
}
```

```

val client = OkHttpClient.Builder()
    .addInterceptor(logging)
    .connectTimeout(30, TimeUnit.SECONDS)
    .readTimeout(30, TimeUnit.SECONDS)
    .build()

val retrofit = Retrofit.Builder()
    .baseUrl("https://jsonplaceholder.typicode.com/")
    .client(client)
    .addConverterFactory(GsonConverterFactory.create())
    .build()

val api = retrofit.create(ApiService::class.java)

// Usage in ViewModel
viewModelScope.launch {
    try {
        val posts = api.getPosts()
        _posts.value = posts
    } catch (e: Exception) {
        _error.value = "Network error: ${e.message}"
    }
}

```

OkHttp (Lower Level)

```

val client = OkHttpClient()

val request = Request.Builder()
    .url("https://api.example.com/users")
    .get()
    .addHeader("Authorization", "Bearer $token")
    .build()

viewModelScope.launch(Dispatchers.IO) {
    val response = client.newCall(request).execute()
    if (response.isSuccessful) {
        val body = response.body?.string()
        // Parse JSON
    }
}

```

WebSocket

```
val client = OkHttpClient()
val ws = client.newWebSocket(
    Request.Builder().url("wss://echo.websocket.org").build(),
    object : WebSocketListener() {
        override fun onOpen(ws: WebSocket, response: Response) {
            ws.send("Hello from Android!")
        }

        override fun onMessage(ws: WebSocket, text: String) {
            println("Received: $text")
        }

        override fun onFailure(ws: WebSocket, t: Throwable, response: Response?) {
            println("Error: ${t.message}")
        }
    }
)
```

Image Loading

```
// build.gradle.kts
// implementation("com.github.bumptech.glide:glide:4.16.0")

// Load image into ImageView
Glide.with(this)
    .load("https://example.com/image.jpg")
    .placeholder(R.drawable.placeholder)
    .error(R.drawable.error)
    .circleCrop()
    .into(imageView)

// Coil (Kotlin coroutines-based alternative)
// implementation("io.coil-kt:coil:2.5.0")

imageView.load("https://example.com/image.jpg") {
    crossfade(true)
    placeholder(R.drawable.placeholder)
    error(R.drawable.error)
}
```

Firestore Realtime Database

```
// build.gradle.kts
// implementation("com.google.firebase:firebase-database")

val database = FirebaseDatabase.getInstance()
val usersRef = database.getReference("users")

// Write
usersRef.child("alice").setValue(User("Alice", 30))

// Read (one-time)
usersRef.child("alice").get().addOnSuccessListener { snapshot ->
    val user = snapshot.getValue(User::class.java)
}

// Read (realtime)
usersRef.child("alice").addValueEventListener(object : ValueEventListener {
    override fun onDataChange(snapshot: DataSnapshot) {
        val user = snapshot.getValue(User::class.java)
        // Update UI
    }

    override fun onCancelled(error: DatabaseError) {
        println("Database error: ${error.message}")
    }
})
```

Connectivity Monitoring

```
val connectivityManager = getSystemService(Context.CONNECTIVITY_SERVICE) as C

// Check current state
val network = connectivityManager.activeNetwork
val capabilities = connectivityManager.getNetworkCapabilities(network)
val isOnline = capabilities?.hasCapability(NetworkCapabilities.NET_CAPABILITY

// Observe changes
val networkCallback = object : ConnectivityManager.NetworkCallback() {
    override fun onAvailable(network: Network) {
        println("Network available")
    }
    override fun onLost(network: Network) {
        println("Network lost")
    }
}
connectivityManager.registerDefaultNetworkCallback(networkCallback)
```

Exercise: Create an app that fetches and displays a list of posts from JSONPlaceholder API using Retrofit. Each item shows the post title. Tapping an item shows the full post in a detail screen. Add pull-to-refresh, error handling with retry, and offline caching using Room as a cache layer.

Multimedia & Graphics

MediaPlayer

```
class AudioPlayer(private val context: Context) {
    private var mediaPlayer: MediaPlayer? = null

    fun playFromUrl(url: String) {
        mediaPlayer = MediaPlayer().apply {
            setDataSource(url)
            setOnPreparedListener { start() }
            setOnErrorListener { _, _, _ -> false }
            prepareAsync()
        }
    }

    fun playFromResource(resId: Int) {
        mediaPlayer = MediaPlayer.create(context, resId)
        mediaPlayer?.start()
    }

    fun pause() {
        mediaPlayer?.pause()
    }

    fun resume() {
        mediaPlayer?.start()
    }

    fun stop() {
        mediaPlayer?.stop()
        mediaPlayer?.release()
        mediaPlayer = null
    }

    fun getCurrentPosition(): Int = mediaPlayer?.currentPosition ?: 0
}
```

Camera API

```
// CameraX (modern camera API)
// build.gradle.kts
// implementation("androidx.camera:camera-camera2:1.3.0")
// implementation("androidx.camera:camera-lifecycle:1.3.0")
// implementation("androidx.camera:camera-view:1.3.0")

class CameraActivity : AppCompatActivity() {
    private lateinit var imageCapture: ImageCapture

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_camera)

        val cameraProviderFuture = ProcessCameraProvider.getInstance(this)
        cameraProviderFuture.addListener({
            val cameraProvider = cameraProviderFuture.get()
            val preview = Preview.Builder().build()
            imageCapture = ImageCapture.Builder()
                .setCaptureMode(ImageCapture.CAPTURE_MODE_MINIMIZE_LATENCY)
                .build()

            val cameraSelector = CameraSelector.DEFAULT_BACK_CAMERA
            cameraProvider.bindToLifecycle(this, cameraSelector, preview, imageCapture)

            // Take photo
            findViewById<Button>(R.id.captureBtn).setOnClickListener {
                val photoFile = File(externalMediaDirs[0],
                    "photo_${System.currentTimeMillis()}.jpg")
                val outputOptions = ImageCapture.OutputFileOptions.Builder(photoFile).build()
                imageCapture.takePicture(outputOptions, ContextCompat.getMainExecutor(this),
                    object : ImageCapture.OnImageSavedCallback {
                        override fun onImageSaved(output: ImageCapture.OutputFileOptions) {
                            Toast.makeText(this@CameraActivity, "Photo saved!", Toast.LENGTH_SHORT).show()
                        }
                    })
            })
        }, ContextCompat.getMainExecutor(this))
    }
}
```

```
}  
}
```

Custom Canvas Drawing

```
class CustomCanvasView(context: Context, attrs: AttributeSet?) : View(context) {
    private val paint = Paint().apply {
        isAntiAlias = true
        style = Paint.Style.FILL
    }

    override fun onDraw(canvas: Canvas) {
        super.onDraw(canvas)

        // Draw background
        canvas.drawColor(Color.WHITE)

        // Draw circle
        paint.color = Color.parseColor("#43a047")
        canvas.drawCircle(200f, 200f, 100f, paint)

        // Draw rectangle
        paint.color = Color.parseColor("#1E88E5")
        canvas.drawRect(50f, 350f, 350f, 500f, paint)

        // Draw text
        paint.color = Color.BLACK
        paint.textSize = 40f
        canvas.drawText("Hello Canvas!", 50f, 600f, paint)

        // Draw line
        paint.strokeWidth = 8f
        paint.style = Paint.Style.STROKE
        canvas.drawLine(50f, 650f, 350f, 650f, paint)

        // Draw path
        val path = Path().apply {
            moveTo(100f, 700f)
            lineTo(200f, 750f)
            lineTo(300f, 700f)
            close()
        }
        paint.style = Paint.Style.FILL
        paint.color = Color.parseColor("#FFC107")
        canvas.drawPath(path, paint)
    }
}
```

```
}  
}
```

Image Effects with Bitmap

```
fun applySepiaEffect(bitmap: Bitmap): Bitmap {  
    val result = bitmap.copy(Bitmap.Config.ARGB_8888, true)  
    val canvas = Canvas(result)  
    val paint = Paint()  
    val sepiaFilter = ColorMatrix(floatArrayOf(  
        0.393f, 0.769f, 0.189f, 0f, 0f,  
        0.349f, 0.686f, 0.168f, 0f, 0f,  
        0.272f, 0.534f, 0.131f, 0f, 0f,  
        0f,    0f,    0f,    1f, 0f  
    ))  
    paint.colorFilter = ColorMatrixColorFilter(sepiaFilter)  
    canvas.drawBitmap(bitmap, 0f, 0f, paint)  
    return result  
}
```

Animations

```
// View animation
view.animate()
    .translationX(200f)
    .translationY(100f)
    .alpha(0.5f)
    .scaleX(1.5f)
    .scaleY(1.5f)
    .rotation(360f)
    .setDuration(1000)
    .setInterpolator(AccelerateDecelerateInterpolator())
    .withEndAction { println("Animation done") }
    .start()

// Lottie animations (JSON-based)
// implementation("com.airbnb.android:lottie:6.3.0")

<com.airbnb.lottie.LottieAnimationView
    android:id="@+id/animationView"
    android:layout_width="200dp"
    android:layout_height="200dp"
    app:lottie_rawRes="@raw/loading_animation"
    app:lottie_loop="true"
    app:lottie_autoPlay="true" />
```

Exercise: Create a photo editor app. Use CameraX to capture a photo, display it in an ImageView, and provide three filters: sepia, grayscale, and invert colors using Bitmap color matrix effects. Add a Lottie loading animation while processing the image.

Sensors & Hardware

Android Sensor Framework

Android devices include various sensors that provide data about the device's environment and motion. Use the `SensorManager` to access them.

```

class SensorActivity : AppCompatActivity(), SensorEventListener {
    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
    }

    override fun onResume() {
        super.onResume()
        accelerometer?.also { accel ->
            sensorManager.registerListener(this, accel,
                SensorManager.SENSOR_DELAY_NORMAL)
        }
    }

    override fun onSensorChanged(event: SensorEvent) {
        if (event.sensor.type == Sensor.TYPE_ACCELEROMETER) {
            val x = event.values[0]
            val y = event.values[1]
            val z = event.values[2]
            println("Accelerometer: x=$x, y=$y, z=$z")
        }
    }

    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}

    override fun onPause() {
        super.onPause()
        sensorManager.unregisterListener(this)
    }
}

```


Sensor Types

Category	Sensors	Use Case
Motion	Accelerometer, Gyroscope, Gravity	Screen rotation, gaming, step counting
Position	Magnetometer, Proximity	Compass, phone call proximity
Environment	Light, Pressure, Temperature, Humidity	Auto brightness, weather

Step Counter

```
class StepCounterService : Service() {
    private lateinit var sensorManager: SensorManager
    private var stepCounter: Sensor? = null

    override fun onCreate() {
        super.onCreate()
        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        stepCounter = sensorManager.getDefaultSensor(Sensor.TYPE_STEP_COUNTER)
    }

    // Register listener
    stepCounter?.let {
        sensorManager.registerListener(listener, it, SensorManager.SENSOR_DELAY_NORMAL)
    }

    private val listener = object : SensorEventListener {
        override fun onSensorChanged(event: SensorEvent) {
            val steps = event.values[0].toInt()
            // Update UI via notification
        }

        override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
    }
}
```

NFC (Near Field Communication)

```
class NFCActivity : AppCompatActivity() {
    private var nfcAdapter: NfcAdapter? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        nfcAdapter = NfcAdapter.getDefaultAdapter(this)

        if (nfcAdapter == null) {
            Toast.makeText(this, "NFC not available", Toast.LENGTH_SHORT).show()
        }
    }

    // Read NFC tag
    override fun onNewIntent(intent: Intent) {
        super.onNewIntent(intent)
        if (NfcAdapter.ACTION_TAG_DISCOVERED == intent.action) {
            val tag = intent.getParcelableExtra<Tag>(NfcAdapter.EXTRA_TAG)
            val id = bytesToHex(tag?.id ?: ByteArray(0))
            println("Tag ID: $id")

            // Read NDEF message
            val ndefMessages = intent.getParcelableArrayExtra(NfcAdapter.EXTRA_NDEF_MESSAGES)
            if (ndefMessages != null) {
                val msg = ndefMessages[0] as NdefMessage
                val record = msg.records[0]
                val payload = String(record.payload)
                println("NDEF: $payload")
            }
        }
    }
}

// Manifest
<uses-permission android:name="android.permission.NFC" />
<intent-filter>
    <action android:name="android.nfc.action.NDEF_DISCOVERED" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:mimeType="*/*" />
</intent-filter>
```

Bluetooth

```
class BluetoothActivity : AppCompatActivity() {
    private val bluetoothAdapter: BluetoothAdapter? by lazy {
        val manager = getSystemService(BLUETOOTH_SERVICE) as BluetoothManager
        manager.adapter
    }

    // Enable Bluetooth
    fun enableBluetooth() {
        if (bluetoothAdapter?.isEnabled == false) {
            val enableIntent = Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE)
            startActivityForResult(enableIntent, REQUEST_ENABLE_BT)
        }
    }

    // Discover devices
    fun discoverDevices() {
        bluetoothAdapter?.startDiscovery()
    }

    val receiver = object : BroadcastReceiver() {
        override fun onReceive(context: Context, intent: Intent) {
            when (intent.action) {
                BluetoothDevice.ACTION_FOUND -> {
                    val device = intent.getParcelableExtra<BluetoothDevice>(B
                    println("Found: ${device?.name} - ${device?.address}")
                }
            }
        }
    }
}
```

Exercise: Create a fitness app that uses the step counter sensor to track daily steps. Show the step count in a notification. When the user reaches 10,000 steps, show a congratulatory notification. Use the light sensor to adjust the theme (light sensor value below threshold = dark mode).

Testing & Debugging

Unit Tests

```
// build.gradle.kts
dependencies {
    testImplementation("junit:junit:4.13.2")
    testImplementation("org.mockito:mockito-core:5.8.0")
    testImplementation("io.mockk:mockk:1.13.8")
}

// ViewModel Unit Test
class UserViewModelTest {
    private lateinit var viewModel: UserViewModel
    private val mockRepo = mock<UserRepository>()

    @Before
    fun setup() {
        viewModel = UserViewModel(mockRepo)
    }

    @Test
    fun `loadUsers should update users LiveData`() = runTest {
        val mockUsers = listOf(User(1, "Alice", "alice@example.com"))
        `when`(mockRepo.getUsers()).thenReturn(mockUsers)

        viewModel.loadUsers()
        val result = viewModel.users.getOrAwaitValue()

        assertEquals(1, result.size)
        assertEquals("Alice", result[0].name)
    }

    @Test
    fun `loadUsers should handle empty response`() = runTest {
        `when`(mockRepo.getUsers()).thenReturn(emptyList())

        viewModel.loadUsers()
        val result = viewModel.users.getOrAwaitValue()

        assertTrue(result.isEmpty())
    }
}
```

```
}  
}
```

UI Tests with Espresso

```
// build.gradle.kts  
androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")  
androidTestImplementation("androidx.test.espresso:espresso-contrib:3.5.1")  
androidTestImplementation("androidx.test.ext:junit:1.1.5")  
  
@RunWith(AndroidJUnit4::class)  
class MainActivityTest {  
    @get:Rule  
    val activityRule = ActivityScenarioRule(MainActivity::class.java)  
  
    @Test  
    fun buttonClick_updatesTextView() {  
        // Find the button and click it  
        onView(withId(R.id.clickMeButton))  
            .perform(click())  
  
        // Check the text view shows expected text  
        onView(withId(R.id.greetingText))  
            .check(matches(withText("Hello, Android!")))  
    }  
  
    @Test  
    fun recyclerView_scrolls() {  
        onView(withId(R.id.recyclerView))  
            .perform(RecyclerViewActions.actionOnItemAtPosition<RecyclerView.ViewHolder>  
                5, click()  
            )  
  
        onView(withText("Item 6"))  
            .check(matches(isDisplayed()))  
    }  
}
```

Mockito & MockK

```
// Mockito (Java-style)
@RunWith(MockitoJUnitRunner::class)
class LoginViewModelTest {
    @Mock lateinit var authRepo: AuthRepository
    lateinit var viewModel: LoginViewModel

    @Before
    fun setup() {
        MockitoAnnotations.initMocks(this)
        viewModel = LoginViewModel(authRepo)
    }

    @Test
    fun login_success() {
        `when`(authRepo.login("user", "pass"))
            .thenReturn(Result.success(User("Alice")))

        viewModel.login("user", "pass")
        assertTrue(viewModel.loginResult.value?.isSuccess == true)
    }
}

// MockK (Kotlin-native)
class MockKTest {
    val mockRepo = mockk<AuthRepository>()

    @Test
    fun loginFailsWithWrongPassword() {
        every { mockRepo.login("user", "wrong") }
            .returns(Result.failure(Exception("Invalid credentials")))

        val result = mockRepo.login("user", "wrong")
        assertTrue(result.isFailure)
    }
}
```

Debugging Tools

```
// Logging
Log.d("MainActivity", "Button clicked at ${System.currentTimeMillis()}")
Log.e("Network", "Request failed: ${error.message}")
Log.i("Lifecycle", "Activity created")

// Android Profiler (in Android Studio)
// View > Tool Windows > Profiler
// Monitor: CPU, Memory, Network, Energy

// StrictMode (catch accidental disk/network on main thread)
if (BuildConfig.DEBUG) {
    StrictMode.setThreadPolicy(StrictMode.ThreadPolicy.Builder()
        .detectDiskReads()
        .detectDiskWrites()
        .detectNetwork()
        .penaltyLog()
        .build())
}

// LeakCanary (memory leak detection)
// implementation("com.squareup.leakcanary:leakcanary-android:2.12")
```

Testing Checklist

Test Type	Framework	What to Test
Unit	JUnit + MockK	ViewModels, Repositories, UseCases
Integration	JUnit + Room	Database, Content Providers
UI	Espresso	Activities, Fragments, Navigation
End-to-End	Compose UI Test	Full user flows

Exercise: Write unit tests for a LoginViewModel using MockK. Mock the AuthRepository and test three scenarios: login succeeds, login fails with wrong password, and login fails with network error. Then write an Espresso UI test that enters credentials, clicks login, and verifies the welcome message appears.

Deployment

Preparing for Release

- Remove all debug logs and `android:debuggable` from manifest
- Update `versionCode` and `versionName` in `build.gradle.kts`
- Run ProGuard/R8 for code shrinking and obfuscation
- Test the release build on a physical device
- Prepare app store assets (icons, screenshots, description)

App Signing

```
// Generate a signing key
// Android Studio: Build > Generate Signed Bundle / APK

// Or via command line:
keytool -genkey -v -keystore my-release-key.jks \
    -keyalg RSA -keysize 2048 -validity 10000 \
    -alias my-alias

// app/build.gradle.kts
android {
    signingConfigs {
        create("release") {
            storeFile = file("my-release-key.jks")
            storePassword = "password"
            keyAlias = "my-alias"
            keyPassword = "password"
        }
    }
    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
            isMinifyEnabled = true
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
}
```

App Bundles vs APK

Google Play requires Android App Bundles (AAB) for new apps. App Bundles allow Google Play to generate optimized APKs for different device configurations.

```
# Generate App Bundle
./gradlew bundleRelease

# Located at: app/build/outputs/bundle/release/app-release.aab

# Generate APK (alternative)
./gradlew assembleRelease

# Test the bundle using bundletool
java -jar bundletool.jar build-apks \
  --bundle=app-release.aab \
  --output=app-release.apks \
  --ks=my-release-key.jks \
  --ks-pass=pass:password \
  --ks-key-alias=my-alias

# Install on device
java -jar bundletool.jar install-apks --apks=app-release.apks
```

Google Play Console

- **Create Developer Account:** One-time \$25 registration fee
- **Dashboard:** View app stats, crashes, ratings
- **Release Management:** Create internal/alpha/beta/production releases
- **Store Listing:** App description, screenshots, feature graphic
- **Pricing & Distribution:** Set pricing, countries, devices
- **In-app Products:** Configure IAP items and subscriptions

Versioning Strategy

```
// app/build.gradle.kts
android {
    defaultConfig {
        applicationId = "com.example.myapp"
        minSdk = 24
        targetSdk = 34

        // Increment versionCode for each release
        versionCode = 5
        versionName = "2.1.0"
    }
}

// Semantic versioning: MAJOR.MINOR.PATCH
// - MAJOR: Breaking changes
// - MINOR: New features (backward compatible)
// - PATCH: Bug fixes

// versionCode vs versionName
// versionCode: Integer, must increment for each update
// versionName: Human-readable version string
```

Crash Reporting

```
// Firebase Crashlytics
// build.gradle.kts
// implementation("com.google.firebase:firebase-crashlytics")

// Initialize in Application class
FirebaseCrashlytics.getInstance().apply {
    setCrashlyticsCollectionEnabled(true)
    log("App started")
    setUserId("user_123")
}

// Log custom events
FirebaseCrashlytics.getInstance().recordException(Exception("Payment failed"))

// Firebase Performance Monitoring
// implementation("com.google.firebase:firebase-perf")
// Automatically monitors app startup, network, and screen rendering
```

CI/CD for Android

```
# .github/workflows/android-ci.yml
name: Android CI

on:
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: 17
      - run: ./gradlew test

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: 17
      - run: ./gradlew bundleRelease
      - uses: actions/upload-artifact@v4
        with:
          name: app-bundle
          path: app/build/outputs/bundle/release/app-release.aab
```

Exercise: Prepare an app for release on Google Play. Generate a signing key, configure the build.gradle.kts for release with ProGuard, create an Android App Bundle, and verify it works by testing via bundletool on a physical device or emulator.

Practice & Resources

Summary of Exercises

1. **Ch1:** Create a new project, add two TextViews and a Button that changes text.
2. **Ch2:** Install Android Studio, create a project, run on emulator.
3. **Ch3:** Build an MVVM app with ViewModel, LiveData, RecyclerView, Repository.
4. **Ch4:** Two Activities with Intent extras and Share button.
5. **Ch5:** Login screen layout with ConstraintLayout and Material Design.
6. **Ch6:** Contact list with RecyclerView, FAB, and Snackbar.
7. **Ch7:** Drawing app with custom View, touch, SeekBar, and swipe detection.
8. **Ch8:** Themed app with custom colors, vector icons, fonts, and Spanish localization.
9. **Ch9:** Notification with action, location updates, and confirmation dialog.
10. **Ch10:** Podcast app with Foreground Service and WorkManager for periodic sync.
11. **Ch11:** Notes app with Room database, RecyclerView, and DataStore.
12. **Ch12:** Posts app with Retrofit, pull-to-refresh, offline caching with Room.
13. **Ch13:** Photo editor with CameraX, bitmap filters, and Lottie animation.
14. **Ch14:** Step counter fitness app with notifications and light sensor.
15. **Ch15:** Unit tests with MockK and UI tests with Espresso.
16. **Ch16:** Prepare app bundle with ProGuard and test with bundletool.

Quiz

1. What are the four main components of an Android app?
2. What is the difference between onStart() and onResume()?
3. How does ConstraintLayout improve performance over nested layouts?
4. What is the purpose of a ViewModel in MVVM?
5. What is the difference between a Service and a BroadcastReceiver?

6. When should you use WorkManager vs a foreground service?
7. How does Room help with SQLite database management?
8. What is the difference between an App Bundle and an APK?
9. What is the purpose of ProGuard/R8?
10. What are the different test types in Android and when should you use each?

Interview Preparation

Common Android Interview Questions

1. Explain the Activity lifecycle in detail.
2. What is the difference between `Serializable` and `Parcelable` ?
3. How does the Android permission system work?
4. Explain the Repository pattern and its benefits.
5. What is dependency injection and why use Hilt/Dagger?
6. How do you handle configuration changes (screen rotation)?
7. What is the difference between `match_parent` and `wrap_content` ?
8. How does RecyclerView recycle views?
9. What is the purpose of LiveData vs StateFlow?
10. How do you handle memory leaks in Android?

Project Ideas

- **Beginner:** Stopwatch app with start/stop/reset and lap recording
- **Beginner:** Currency converter app with live exchange rates
- **Intermediate:** Recipe app with Room, Retrofit, and image loading
- **Intermediate:** Fitness tracker with step counter, maps, and charts
- **Advanced:** Chat app with Firebase Auth, Firestore, and push notifications
- **Advanced:** Expense tracker with Room, WorkManager, and data export

Study Plan

Week	Topics	Goal
1	Ch1-4	Understand Android fundamentals and components
2	Ch5-8	Master UI design and resources
3	Ch9-12	Networking, storage, and services
4	Ch13-16	Multimedia, sensors, testing, deployment
5	Ch17-18	Build a complete app and prepare for interview

Certification Path

- **Google Associate Android Developer (AAD):** Official Google certification
- **Meta Android Developer Certificate:** Meta's Android development program
- Complete 3 portfolio apps and publish one to Google Play
- Contribute to open-source Android projects

Final Project: Build a "Expense Tracker" Android app with Room database, Retrofit for currency conversion, WorkManager for monthly backup, MVVM architecture, Hilt DI, push notifications for spending alerts, a custom view for pie charts, and at least 80% test coverage. Publish it to Google Play as an internal test.

References

Android API Reference

Package / Class	Purpose
android.app.Activity	Single screen UI with lifecycle management
android.app.Fragment	Reusable portion of UI
android.app.Service	Background operations without UI
android.content.BroadcastReceiver	Receive system broadcast events
android.content.ContentProvider	Share data between applications
android.content.Intent	Inter-component communication message
android.view.View	Base class for UI widgets
android.widget.TextView	Text display widget
android.widget.RecyclerView	Efficient list/grid display
android.database.sqlite.SQLiteDatabase	SQLite database management

Glossary

Term	Definition
Activity	A single, focused task the user can do
ADB	Android Debug Bridge — CLI tool for device communication
APK	Android Package Kit — app installation file
ART	Android Runtime — runs app bytecode
AAB	Android App Bundle — publishing format
DP / DIP	Density-independent pixel
Fragment	A portion of UI in an Activity
Gradle	Build automation system for Android
Intent	Message to request an action from another component
MVP / MVVM	Architecture patterns for organizing code
ProGuard	Code shrinker and obfuscation tool
SDK	Software Development Kit
SP	Scale-independent pixel (for fonts)
ViewModel	Stores UI-related data across config changes

Useful Resources

- **developer.android.com**: Official Android documentation, guides, and API reference
- **developer.android.com/kotlin**: Kotlin for Android
- **developer.android.com/jetpack**: Android Jetpack libraries
- **material.io**: Material Design guidelines and components
- **android-arsenal.com**: Curated directory of Android libraries and tools
- **kotlinlang.org**: Official Kotlin language documentation

- **medium.com/androiddevelopers**: Official Android Developers blog
- **github.com/android**: Official Android samples on GitHub
- **stackoverflow.com/questions/tagged/android**: Android Q&A community
- **codelabs.developers.google.com**: Google Codelabs with step-by-step tutorials

Recommended Libraries

Library	Purpose
Retrofit	HTTP client API
Glide / Coil	Image loading
Room	SQLite ORM
Hilt / Dagger	Dependency injection
Moshi / Gson	JSON serialization
Jetpack Compose	Modern declarative UI toolkit
Navigation Component	Screen navigation
DataStore	Modern key-value storage
WorkManager	Deferrable background tasks
Lottie	JSON-based animations

Congratulations! You've completed the Android Tutorial: A Complete Guide. Start building, keep experimenting, and publish your first app to Google Play!